Intaliq

SE322: Software Design and Architecture

Section: 1340

Instructor

Dr. Suad Ramouni

Prepared by:

YARA FARIS FAIHAN ALBUGAMI

ROSE SAEED RAKAN AL RAKAN

SHOUG FAWAZ ABDULLAH ALOMRAN

FAJR ALGHOFAILI

MOZOUN ALMUQRIN

MUNIRA YAZEED ABDULAZIZ ALFARIS

Table Of Contents

Deliverable 1: **5**

1. Introduction **5**

1.1 Overview 5

1.2 Purpose 5

1.3 Scope 6

1.4 Definitions, acronyms, and abbreviations 7

Session class type 7

User class type 8

Coach class type 8

Goal class type 9

Partner class type 9

Activity class type 10

1.5 References 10

2. Architectural Representations **10**

3. Architectural Goals and Constraints **10**

4. User View 11

4.1 Use case 11

4.2 Scenarios 11

4.2.1 Book a session 11

4.2.2 Communicate with AI Chatbox 12

4.2.3 Enter activity goal 13

5. Logical View **14**

5.1 Class diagram 14

5.2 Sequence Diagrams 15

Logging an Activity 15

Setting an Activity Goal 16

Booking a Group Session 16

Creating a Group Session 17

Admitting Members 17

6. Process View **17**

6.1 Activity diagram 18

7. Development View **18**

7.1 Component diagram 18

8. Physical View **19**

8.1 Deployment diagram 19

Deliverable 2: **20**

1. Quality Attributes **20**

1.1 Quality Attribute: Security 20

1.2 Quality Attribute: Availability 20

1.3 Quality Attribute: Performance 20

1.4 Quality Attribute: Usability 21

2. Quality Attribute Scenarios **21**

2.1 Scenario for Quality Attribute: Security....21
2.2 Scenario for Quality Attribute: Availability....21
2.3 Scenario for Quality: Performance....22
2.4 Scenario for Quality Attribute: Usability....22
3. Architectural Style....22
3.1 Selected Architectural Style....22
3.2 Justification....23
3.3 How We Will Implement It in Intaliq....24
1. Client: Intaliq Mobile Application....24
2. Application Server: Intaliq Backend....24
3. Database: Intaliq Database....24
4. External AI Service....24
Deliverable 3:....25
1. Introduction....25
1.1 Date of Issue....25
1.2 Context....25
1.3 Scope....26
1.4 Authorship....26
1.5 Change history....26
1.6 Summary....27
2. Software Architecture....27
2.1 Overview....27
2.2 Stakeholders....28
2.3 System Design Concerns....28
2.4 Client-Server Architectural Viewpoint....29
2.4.1 Client-Server Design View....29
Deployment Diagram....30
2.4.2 Justification....30
2.5 Mobile Application Architectural Viewpoint....31
2.5.1 MVC Design View....31
MVC Design Diagram....32
2.5.2 Justification....32
2.6 Backend Architectural Viewpoint....33
2.6.1 Layered Architecture Design View....33
Layered Architecture Design Diagram....33
2.6.2 Justification....33
3. Detailed Design....33
3.1 Overview....33
3.2 Factory Method (Creational)....34
3.2.1 How It Is Applied....34
3.2.2 Justification....35
3.3 Observer (Behavioral)....35
3.3.1 How It Is Applied....35
3.3.2 Justification....36

3.4 Adapter (Structural)..... 37
3.4.1 How It Is Applied..... 37
3.4.2 Justification..... 37
4. Reference..... 38
5. Glossary..... 38

Deliverable 1:

1. Introduction

1.1 Overview

Intaliq aims to provide a solution to multiple issues Saudi Arabians are facing with outdoor activities currently, including the following:

- Monitoring outdoor activity: There is a need for fitness tools that accurately monitor users'outdoor activity levels.
- Finding partners: A lot of people struggle with finding partners to join them in their preferred outdoor activity.
- Joining Group Sessions: It is hard to find big groups for larger scale activity events that are led by professionals in the field especially in Saudi Arabia.

The Kingdom of Saudi Arabia is undergoing a bold transformation as part of Vision 2030, which aims to strengthen the quality of life with a specific emphasis on well-being and health of its citizens. The government has set visionary goals, one of the main goals being to increase the life expectancy average from 74 years to 80 years and significantly jump in the population's rate of weekly exercise from 13% to 40%. These goals highlight the importance of physical activity as a key factor in reducing chronic diseases. This is where our app Intaliq comes into play, empowering citizens to adopt a more active and healthier lifestyle.

1.2 Purpose

Intaliq is a fitness mobile application created to make outdoor activities, such as running, cycling, and hiking, more accessible and enjoyable in Saudi Arabia. It allows users to track their daily activity level, set goals for themselves, and join other users in group sessions. This software helps guide users through outdoor activities by providing a trained team of professionals that hold daily classes. Intaliq has a simple intuitive design that makes it easy for users to navigate routinely, as well as aids outdoor activities appear less intimidating.

The primary objectives of this system are:

- Assist users with navigating outdoor activities in Saudi Arabia by delivering an easy-to-use interface.
- Provide accurate tools for monitoring and setting activity goals.
- Help users make social connections and build relationships by creating a platform that connects people all over Saudi Arabia who have an interest in outdoor activity.

By combining these features into a single app, Intaliq promotes not only personal wellness but also a healthier society and community. It supports the government objectives to promote proactive and preventive healthcare practices, encourages the population in regular physical activities and exercise, and creates a community that values and embraces fitness and well-being . Motivated by Vision 2030, Intaliq strives to play a significant role in evolving and turning Saudi Arabia into a healthier, more active society with an improved lifespan and well-being.

1.3 Scope

The Intaliq system is a fitness software for smart phones that was created to promote outdoor exercises in Saudi Arabia. Users are able to log activities such as running, cycling, and hiking, set personal fitness goals, and take part in fitness coach-led group sessions.

Through social connections and organized activities, the system seeks to increase the accessibility, motivation, and social engagement of outdoor fitness.

Intaliq has a variety of features that are built to aid Saudi citizens in navigating outdoor activities. Some of the key features include:

- Activity Tracking: The system allows users to log their daily outdoor activities and track improvements.
- Activity Goal Setting: Users can set and adjust their activity goals to monitor progress.
- AI Analysis: Users can get an overview of their improvements by chatting with an AI chatbot or make adjustments to their goals.
- Finding Partners: The system provides a feature that allows users to chat and find partners to help motivate each other.
- Joining large groups for sessions: Users can locate and join a big group in large scale sessions, such as: running crew, hiking or cycling groups, hosted by Intaliq's own fitness team.
- Creating Group Sessions: Coaches from Intaliq's fitness team can create and schedule a new group session for users to join.
- Admitting Group Members: A Coach can accept users' applications to join sessions, and admit them if their goals and current level matches with the session's objectives.
- Announcing Session Updates: The system allows coaches to make announcements about their scheduled sessions ahead of time, and any changes that may occur.

1.4 Definitions, acronyms, and abbreviations

Session class type		
Data Name	**Data type**	**Constraint**
date	string	- By default, must follow the template: DD/MM/YYYY

Session class type		
		- Can’t be an unrealistic date (far future/past)
time	string	- By default, must use the 24-hour clock time
location	string	- Must be located in Saudi Arabia - Must be permitted for group physical activities
capacity	int	- Must be more than 1 - Depending on event type and scale, location must accomodate
intensity	string	- Must fall under beginner, intermediate, or advanced

User class type		
Data Name	**Data type**	**Constraint**
firstName	string	- Must be less than 20 characters
lastName	string	- Must be less than 35 characters
phoneNumber	string	- Saudi Arabian phone number must follow the template: +966 5X-XXX-XXXX - Non-Saudi Arabian phone numbers must have their country code followed by their number
email	string	- Domain must be officially recognized - Email handle must already exist - Email handle should be less than 64 characters
password	string	- Must be 8 characters minimum with 1 upper case, 1 number, and 1 special character

Coach class type		
Data Name	**Data type**	**Constraint**
firstName	string	- Must be less than 20 characters
lastName	string	- Must be less than 20 characters

Coach class type		
phoneNumber	string	- Saudi Arabian phone number must follow the template: +966 5X-XXX-XXXX - Non-Saudi Arabian phone numbers must have their country code followed by their number
email	string	- Domain must be officially recognized - Email handle must already exist - Email handle should be less than 64 characters
password	string	- Must be 8 characters minimum with 1 upper case, 1 number, and 1 special character

Goal class type		
Data Name	**Data type**	**Constraint**
type	string	- Follows the 3 activities that Intaliq focuses on: Hiking, Biking, and Running
targetTime	string	- Must follow template HH:MM
caloriesBurnt	string	- Must be more than 10 kCal
deadline	string	- By default, must follow the template: DD/MM/YYYY - Can’t be an unrealistic date (past)

Partner class type		
Data Name	**Data type**	**Constraint**
location	string	- Must be located in Saudi Arabia - Must be permitted for group physical activities
goals	string	- Must relate to the 3 activities that Intaliq focuses on: Hiking, Biking, and Running - Other constraints mentioned in goal class must be noted
activities	string	- Follows the 3 activities that Intaliq focuses on: Hiking, Biking, and Running

Activity class type		
Data Name	**Data type**	**Constraint**
type	string	- Follows the 3 activities that Intaliq focuses on: Hiking, Biking, and Running
duration	string	- Must be 5 minutes and more - Shouldn't exceed a day

1.5 References

- Intaliq Vision and Scope Document
- IEEE Software Requirements Specification (SRS) Template

2. Architectural Representations

We use the 4+1 View Model, introduced by Philippe Kruchten in 1995, to describe the architecture of Intaliq. Since Intaliq has different parts that need to be considered from different perspectives, we used the model to break the architecture into five related views. The system is mainly used by regular users and coaches, while developers are responsible for building and maintaining the application. Looking at the system from these different perspectives makes it easier to explain how its components work together. The model consists of four main views: Logical, Process, Development, and Physical, along with the User View (the "+1"). The User View is based on use cases and scenarios and helps connect the other views to the way users interact with the system. The following sections describe each view and how it applies to Intaliq. Using the 4+1 View Model also gives us a clear way to organize the architecture and show how the different parts of Intaliq fit together.

3. Architectural Goals and Constraints

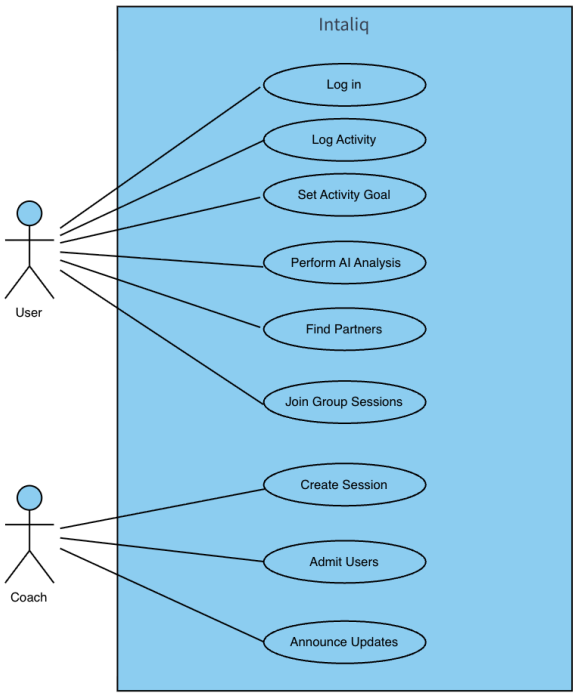
The architecture of intaliq has goals that align with its purpose of encouraging Saudi citizens to be physically active and connect with others through sports. Some of the goals associated with the architecture are:

- providing an easy-to-use interface on mobile phones
- the protection of user accounts and keeping their information safe, including its integrity and confidentiality.

Also, some constraints will limit the implementation and architecture of the app, including the language. In the project, JavaScript will be used to develop the application. Other limitations, like time and cost for this project, will be discussed later

4. User View

4.1 Use case



4.2 Scenarios

4.2.1 Book a session	
Actors	Users

4.2.1 Book a session	
Description	A user can browse the available group sessions and book a session to take part in outdoor exercises, such as cycling, hiking, or running. The user can view available sessions, check session details, including the coach, and submit a request to join the session.
Data	User information. Session details: activity type, time, date, location, capacity of a session, intensity level of a session, and coach information.
Stimulus	User command on application selects and books a session
Response	Confirmation message that the booking request has been submitted. If accepted by the coach, the user is added to the session. If rejected, the user is notified.

Normal Scenario:

1. User opens the Intaliq app
2. User logs into the system
3. User browses available group sessions
4. User select session
5. User the review session details and coach details
6. User submits a booking request
7. The coach reviews the request
8. The coach accepts the request
9. The system confirms the booking and notifies the user

Alternative Scenario:

- If the user is not logged in —> system prompt login with a pop up message
- If the session is full —> the system prevents sending the request with a pop-up message
- If the coach rejects the request —> the system notifies the user
- If a system error occurs —> the request is not completed and the user is notified

4.2.2 Communicate with AI Chatbox	
Actors	Users

4.2.2 Communicate with AI Chatbox	
Description	A user can chat with an AI Chatbox for fitness related advice.
Data	User entered-query
Stimulus	User command on AI chatbox chat
Response	AI Chatbox will answer user accordingly and suggest further help.

Normal Scenario:

1- User opens the Intaliq app

2- User logs into the system

3- User selects more tab

4- User select AI Chatbox

5- User select from given chat options or write their own

6- User submits the chat option by clicking

7- AI coach will answer the chosen query

8- User accepts answer and closes chat

Alternative Scenario:

- If the user is not logged in —> system prompt login with a pop up message
- If user asks for medical advice that should be only given from trained specialists —> AI chatbox can refuse answer and end chat
- If a system error occurs —> the chat request is not completed and the user is notified

4.2.3 Enter activity goal	
Actors	Users.
Description	A user can enter an activity goal they want to achieve.
Data	User information, activity type, specific goal to achieve.
Stimulus	User command on application.
Response	Confirmation by the system that the goal has been saved.

Normal scenario:

1- The user opens the Intaliq app.

2- The user logs into the system.

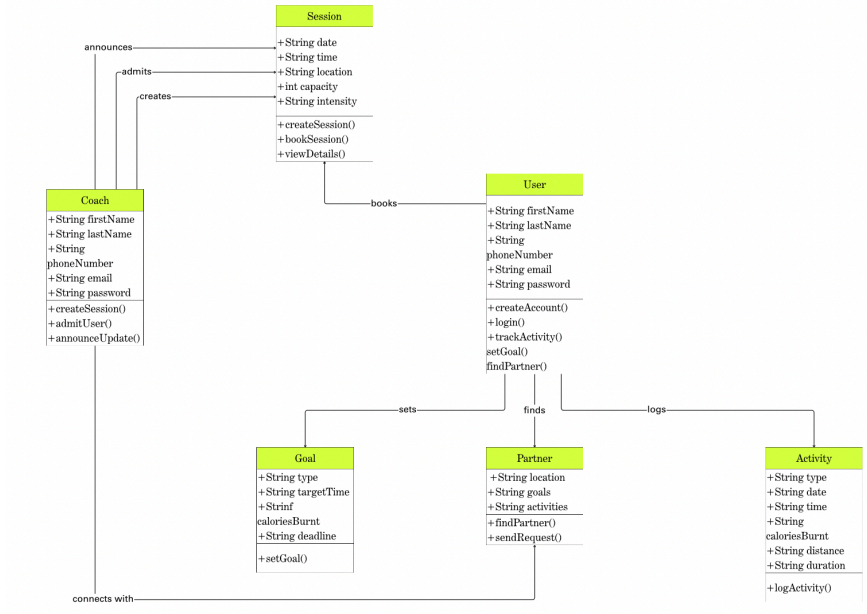
3- The user selects “Activity”.	5- The user submits the goal.
5- The user selects “Activity Goal”.	6- The system confirms the input.
4- The user enters their activity goal (activity type, goal).	7- The system saves the goal.
	8- The system confirms the goal has been successfully created.

Alternative Scenario:

- If the user is not logged in —> system prompt login with a pop up message
- If any required field is missing —> The system prompts the user to fill in the missing information.
- If invalid data is entered —> The system displays an error message.
- If a system error occurs —> The goal is not saved.

5. Logical View

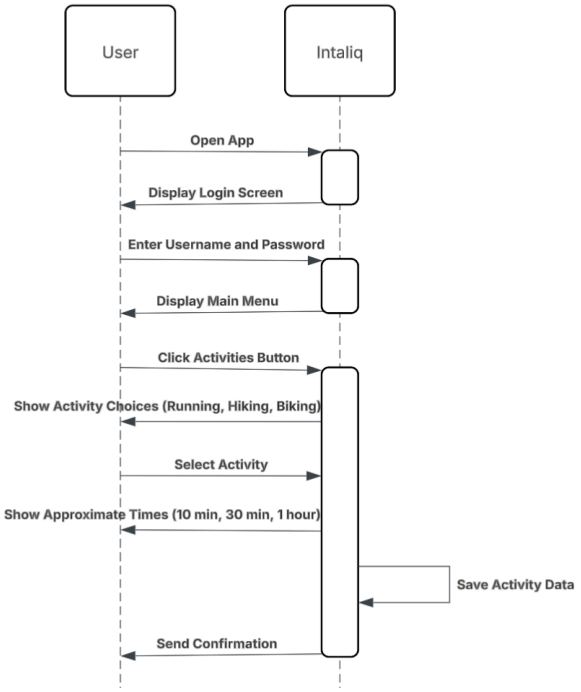
5.1 Class diagram



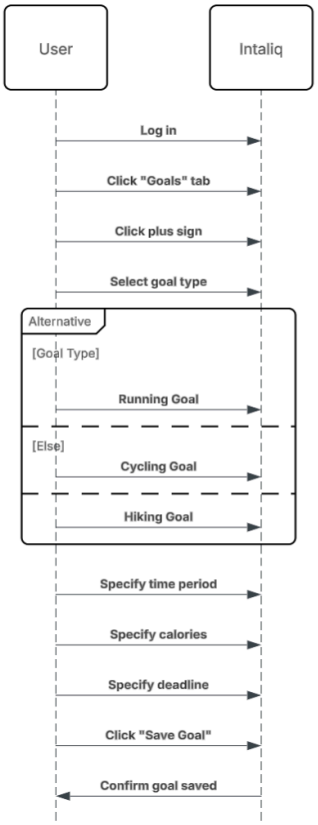
Our class diagram includes the following classes: Session, User, Goal, Partner, Activity, and Coach. Users can start goals, and log their activities. They can also find partners, and even book sessions created by coaches. Coaches, based on their judgement, can either admit or reject users into a session. To get more awareness, coaches can announce sessions as well. Lastly, coaches can connect with partners.

5.2 Sequence Diagrams

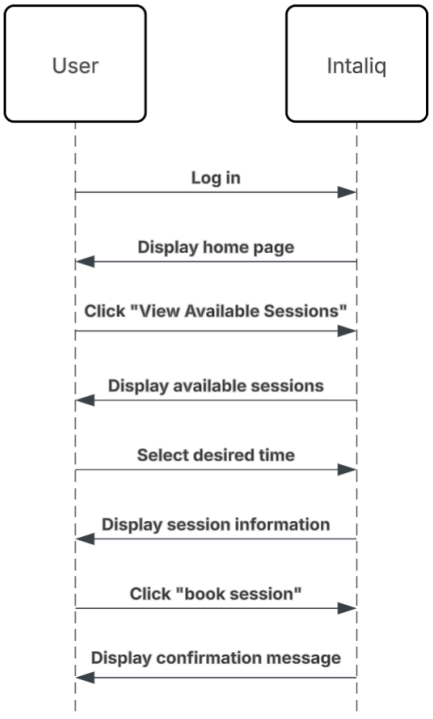
Logging an Activity



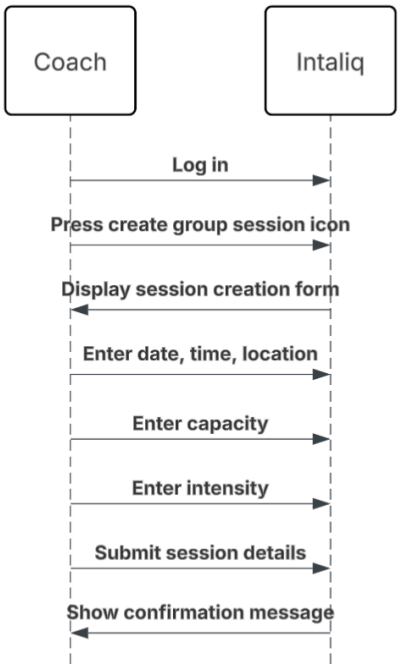
Setting an Activity Goal



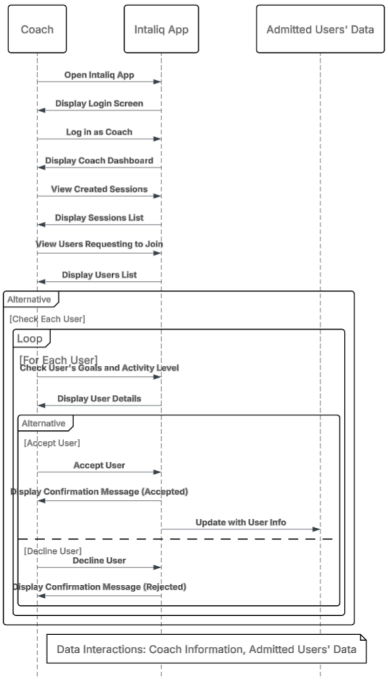
Booking a Group Session



Creating a Group Session



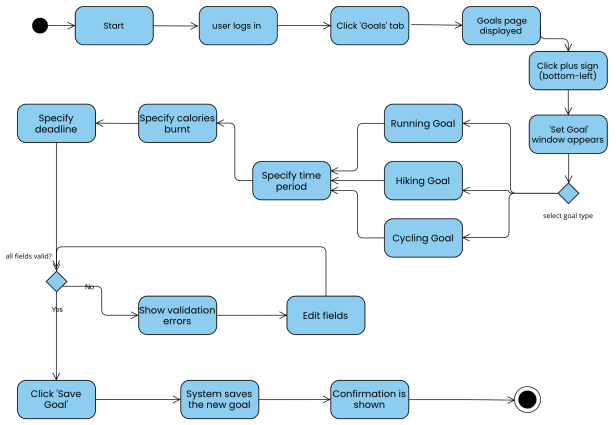
Admitting Members



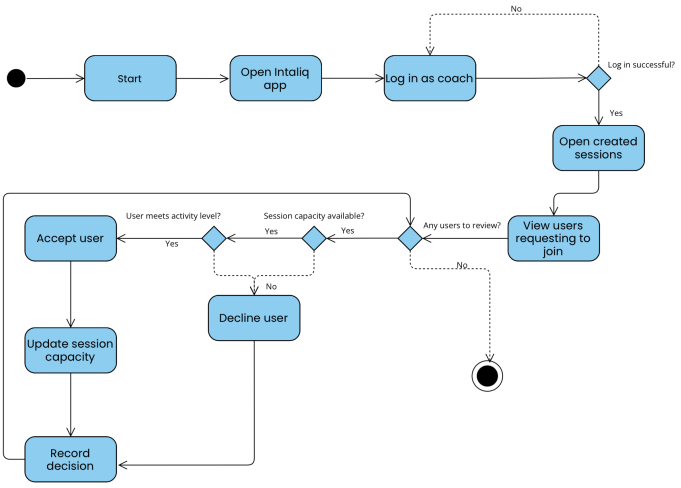
6. Process View

6.1 Activity diagram

Activity diagram for Setting an Activity Goal (user)

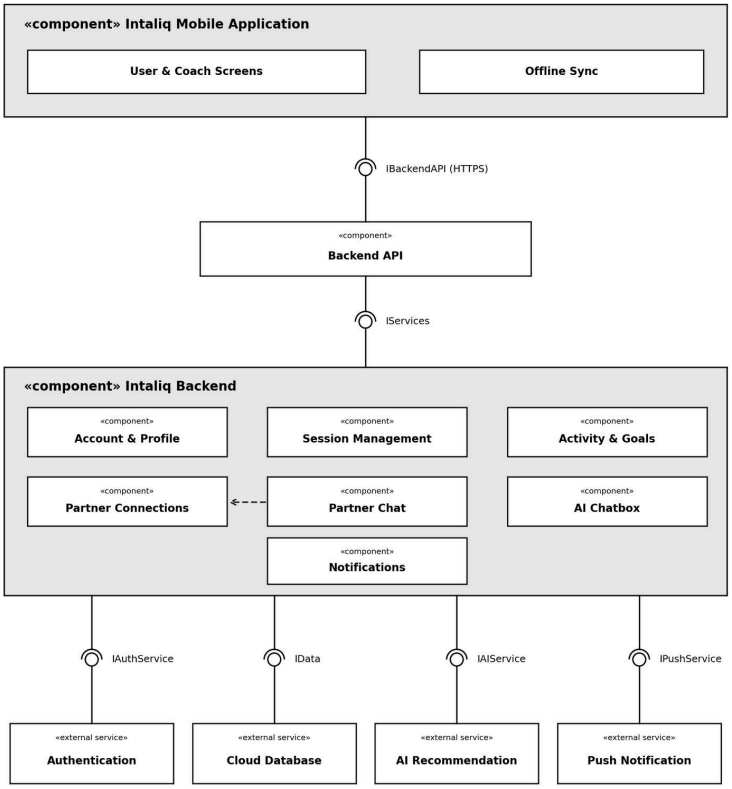


Activity diagram for Admitting Members (coach)



7. Development View

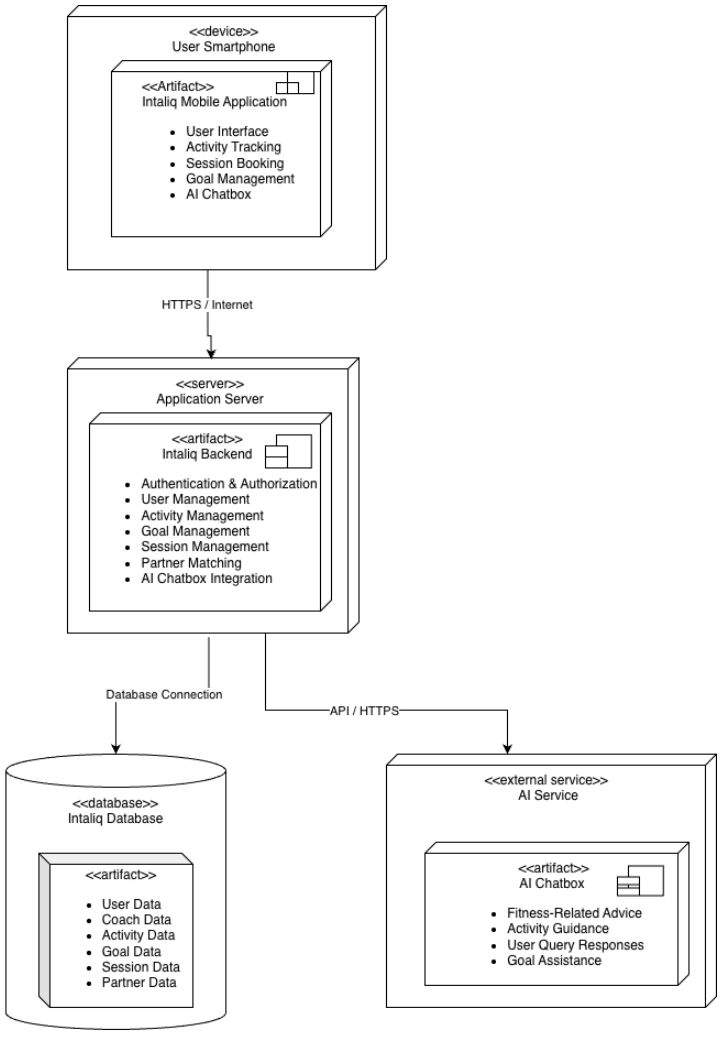
7.1 Component diagram



The component diagram displays the primary software components within Intaliq as well as their interfaces. Communication between the Mobile Application and the Backend API is via HTTPS. Within the Backend, there are multiple components that handle various tasks including Account & Profile Management, Sessions, Activities & Goals, Partner Connections, Partner Chat, AI Chat, and Notifications. The Backend interacts with external Authentication, Cloud Database, AI Recommendation, and Push Notification Services. Partner Chat is only accessible after a Partner Connection has been established, whereas Offline Sync stores all actions temporarily until connected back to the Internet.

8. Physical View

8.1 Deployment diagram



Deliverable 2:

1. Quality Attributes

1.1 Quality Attribute: Security

One of the key quality attributes in Intaliq is *security*, as the application contains private data such as user account data, logs, and personal fitness goals. The system must ensure that the data is protected from any unauthorized access and the users have to be authenticated before accessing their accounts.

The authentication and authorization mechanisms in Intaliq must ensure that both coaches and regular users are restricted to execute only those functions for which they are authorized.

1.2 Quality Attribute: Availability

Availability is essential as users will have to access Intaliq in order to monitor their activities, define their fitness objectives, find partners, and make appointments for group classes.

Intaliq should be available throughout regular operation; however, in case any of its services becomes unavailable, the application should warn users and ensure that incomplete procedures do not get considered as completed.

Moreover, the Intaliq architecture features the Offline Sync functionality which holds actions till the time the device gets online again.

1.3 Quality Attribute: Performance

Why *performance* matters: Users of the Intaliq software must be able to use the program in an efficient manner. This means that tasks such as checking available sessions, logging, and recording fitness goals should be done in a timely manner without any significant delay.

Efficiency in processing user input through the mobile app and the back end server is necessary. Good performance will ensure that users can use the application efficiently.

1.4 Quality Attribute: Usability

Usability is necessary since the purpose of the program is to motivate the citizens of Saudi Arabia to engage in outdoor fitness exercises.

The program must ensure the ease of use through the provision of a user-friendly mobile interface whereby users can access all the elements of the application without going through complex steps.

Moreover, the program must have an alert system for successful completion of actions.

2. Quality Attribute Scenarios

2.1 Scenario for Quality Attribute: Security

Element	Description
Source	An unauthorized user
Stimulus	Attempts to access another user's fitness information without authentication
Environment	Normal system operation
Artifact	Authentication and Authorization component
Response	The system verifies the user's authentication and permissions and denies unauthorized access
Response Measure	100% of unauthorized access attempts in the defined security test cases are rejected

2.2 Scenario for Quality Attribute: Availability

Element	Description
Source	A user experiencing an internet connection failure
Stimulus	The user attempts to log an activity while the internet connection is unavailable

Environment	Mobile application operating without internet access
Artifact	Intaliq Mobile Application and Offline Sync component
Response	The system temporarily stores the activity locally and synchronizes it when the internet connection is restored
Response Measure	100% of successfully queued test activities synchronize after reconnection without data loss

2.3 Scenario for Quality: Performance

Element	Description
Source	An authenticated user
Stimulus	Requests to view available group sessions
Environment	Normal system operation with an active internet connection
Artifact	Intaliq Mobile Application, Backend API, and Database
Response	The system retrieves the available sessions and displays their details
Response Measure	95% of session-list requests complete within 3 seconds under the agreed normal-load test conditions

2.4 Scenario for Quality Attribute: Usability

Element	Description
Source	A new Intaliq user
Stimulus	Attempts to create a personal fitness goal
Environment	Normal mobile application operation
Artifact	Intaliq Mobile Application user interface
Response	The application guides the user through selecting an activity, entering a goal, and saving it with a confirmation message
Response Measure	At least 90% of first-time test users successfully create a fitness goal without assistance

3. Architectural Style

3.1 Selected Architectural Style

In Intaliq, Client Server architecture is implemented, wherein the mobile application will be the client communicating with a centralized backend server.

The client will provide the user interface for users to interact with the application's features such as activity monitoring, goal setting, session booking, partner matching, and AI Chatbox.

The application server will take care of user interaction with the system and processes requests; handles authentication and authorization; application level operations; and communicates with the database and other external services.

The database will be used to store information about users, coaches, activities, goals, sessions, and partners. The backend will communicate with an external AI service to enable the features of AI Chatbox.

3.2 Justification

Client-Server Architecture was chosen as a design strategy due to its clear division of mobile application and system's backend processes.

Here are some factors supporting the use of Client-Server Architecture for Intaliq:

- Data Centralization: User information, user actions, fitness goals and sessions can be controlled via the backend and the database.
- Security: Authentication and authorization processes can be performed on the server to prevent access to user information.
- Maintability: Backend and mobile applications can be maintained independently from each other.
- Performance: Data processing and database interactions will be done by the backend while the mobile application will interact with users.
- Integration: The architecture allows performing data exchange with third-party services such as AI Chatbox.

In general, Client-Server Architecture allows organizing the Intaliq system in a logical way.

3.3 How We Will Implement It in Intaliq

Implementation of Client-Server Architecture in Intaliq will be based on four key components:

1. Client: Intaliq Mobile Application

This is the application that will enable interaction of users with the system by performing the actions of logging their activities, setting goals, booking sessions, finding partners and communication with the AI Chatbox.

2. Application Server: Intaliq Backend

Backend receives requests for operations performed by the mobile application via HTTPS protocol. These operations include:

- Authentication and Authorization
- User Management
- Activity Management
- Goal Management
- Session Management
- Partner Matching
- Integration with the AI Chatbox

3. Database: Intaliq Database

This database stores the information needed for the application operations including:

- User Information
- Coach Information
- Activity Information
- Goal Information
- Session Information
- Partner Information

4. External AI Service

The back end interacts with the external AI service through an API connection that utilizes the HTTPS protocol. The AI service provides support for the AI Chatbox and enables the user to ask questions and get fitness tips.

Communication Process

1. User executes an action via the Intaliq Mobile Application.
2. The application passes on the request to the back end server through HTTPS.
3. The back end authenticates the request and processes it as necessary.
4. The back end communicates with the database or the external AI service when required.
5. The back end responds to the application with the outcome of the process.
6. The application shows the outcome to the user.

The above design is in line with the component and deployment diagrams discussed in Intaliq's Software Architecture Design document.

Deliverable 3:

1. Introduction

1.1 Date of Issue

The Software Design Document was done on October 25, 2026

1.2 Context

Intaliq is a fitness app for mobile phones that facilitates engagement in outdoor sports like running, cycling, and hiking in the Kingdom of Saudi Arabia. This platform caters to the objectives of the Vision 2030 program by helping achieve more exercise participation weekly and enhanced quality of life. Through this platform, one is able to register activities, set goals, connect with partners, attend coach-led sessions, and receive fitness tips through an AI chatbox.

This document builds on earlier work on the project:

- SE311's Software Requirements Specification (SRS), that provided functional and non-functional requirements.

- The high fidelity prototype from SE365 that was made using JavaScript, Vite, Supabase, and Vercel.
- Software Architecture Design (SAD) from Deliverable 1 that used 4 + 1 View Model to model the system.
- Architecture Documentation from Deliverable 2 that had four quality attributes (Security, Availability, Performance, and Usability) and Client-Server architectural style.

The SDD moves from these architecture-level decisions down to the design level by applying design patterns to Intaliq's components.

1.3 Scope

Intaliq is a fitness software that was created to promote outdoor exercises in Saudi Arabia. Users can log activities such as running, cycling, and hiking, set personal fitness goals, and take part in fitness coach-led group sessions. Coaches can also benefit by promoting their sessions to reach a broader audience. Through social connections and organized activities, the system seeks to increase the accessibility, motivation, and social engagement of outdoor fitness.

1.4 Authorship				
Prepared by	**Project Name**	**Course**	**Institution**	**Document Type**
Yara Albugami Rose Al Rakan Shoug Alomran Fajr Alghofaili Mozoun Almuqrin Munira Alfaris	Intaliq	SE322: Software Design And Architecture	Prince Sultan University	SDD: Software Design Description

1.5 Change history			
Version	**Date**	**Deliverable**	**Description of Changes**

1.5 Change history			
1.0	30/09/2026	Deliverable 1: SAD	Initial architecture using the 4+1 View Model, including use case, class, sequence, activity, component, and deployment diagrams.
2.0	10/10/2026	Deliverable 2: Architecture Documentation	Added four quality attributes with scenarios and selected the Client-Server architectural style.
3.0	25/10/2026	Deliverable 3: SDD	Added design patterns, their justification, and how they apply to Intaliq.

1.6 Summary

This SDD outlines the implementation of Intaliq's Client Server Architecture by means of design patterns. Introduction is provided in Section 1, where background, scope, and the authorship of the document are introduced. The subsequent sections outline the design pattern, where it is applied in Intaliq and the rationale behind the selection of that particular pattern. These patterns assist in achieving the objectives of Intaliq in terms of security, availability, performance, and usability.

2. Software Architecture

2.1 Overview

Intaliq follows a Client-Server architecture, as selected in Deliverable 2. The system has four main parts:

- Intaliq Mobile Application (Client): The client through which the user and coach interact with logging of activities, goal setting, scheduling sessions, finding partners, and using the AI Chatbox.
- Intaliq Backend (Application Server): This is the server that will accept requests by the mobile application using HTTPS protocol. Authentication, user, activities and

goals, sessions, partner finding, notification and AI chatbox will be handled by the server.

- Intaliq Database: This holds the user, coach, activities, goals, sessions, and partner information.

The design patterns in this document are applied mainly inside the Intaliq Backend, where the business logic and external integrations are handled.

2.2 Stakeholders	
Stakeholder	**Description**
End Users	People who do outdoor running, hiking, or cycling, from beginners to experienced
Coaches	Intaliq's fitness team who create and manage group sessions
Development Team	The people who build the system from this design
Testers	The people who check that the system behaves as specified
Maintainers	The people who update and extend the system after release

2.3 System Design Concerns		
No.	**Stakeholder**	**Concerns to Address**
1	Development Team, Maintainers	Adding new activity types without changing existing code
2	End Users, Coaches	Receiving timely updates about sessions and booking decisions
3	Development Team, Product Owners	Using external services without depending on one provider
4	Testers	Testing each part of the system on its own
5	End Users	Keeping personal data secure

2.4 Client-Server Architectural Viewpoint

This viewpoint shows Intaliq's overall structure: how the system is divided between the client and the server, and how they communicate. It addresses concerns 4 and 5 by separating the system into independent parts and keeping all data access behind the server.

2.4.1 Client-Server Design View

In Intaliq, there is a client-server pattern; hence, Intaliq can be considered a Distributed System. There are two major parts of the system: the client and the server. The client refers to the part of the program that works with users, while the server is used for processing requests made by users. There are many clients (both users and coaches) that communicate with one server only; there is no direct communication between them.

Primary presentation: deployment diagram showing the User Smartphone running the Intaliq Mobile Application, connected over HTTPS to the Application Server running the Intaliq Backend, which connects to the Intaliq Database and the External AI Service.

Element catalog		
Element	**Role**	**Responsibility**
Intaliq Mobile Application	Client	Displays screens for users and coaches, sends requests to the server, and stores actions locally through Offline Sync when the connection is lost
Intaliq Backend	Server	Receives requests over HTTPS, checks authentication and permissions, runs the business logic, and returns results
Intaliq Database	Data store	Stores user, coach, activity, goal, session, and partner data
External AI Service	External service	Answers fitness questions for the AI Chatbox, reached by the server through an API over HTTPS

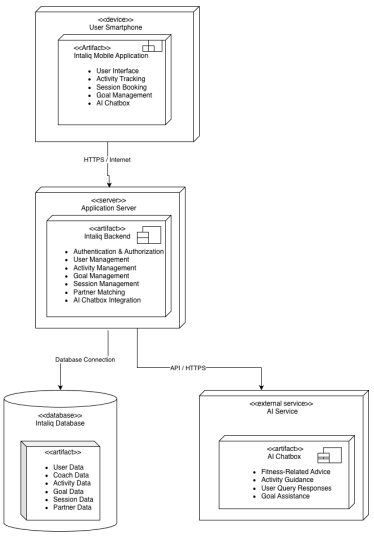
Communication flow:

1. The request is made by the user from a particular function in the mobile application, say, making an appointment.
2. The request is passed to the backend through HTTPS.

3. The backend verifies and processes the request.
4. The backend reads from or writes to the database, or calls the external AI service when needed.
5. The backend returns the result to the application.
6. The application displays the outcome to the user.

Rationale: Client-Server fits Intaliq because all users and coaches share the same data, such as sessions and partner requests, which must stay consistent in one central place. Keeping authentication on the server supports **security**, since the client never accesses the database directly. Server changes reach every client at once, which supports **modifiability**, and the server can be backed up to support **availability**. Broker was considered but rejected, since Intaliq has a single backend and doesn't need a mediator between multiple servers.

Deployment Diagram



2.4.2 Justification

In Intaliq, users and coaches work with the same information at the same time. When a coach creates a session, every user must see the same details and the same number of available spots. As stated in the "Book a Session" scenario, the coach also needs to accept or decline the booking request, and the user must see the result. If each mobile application kept

its own copy of the data, the information could become different between users, and two people might book the last spot.

Client-Server was the best choice for Intaliq because the data has to be shared. In this architecture, the mobile application is just the part the user sees. It shows the screens and sends requests over HTTPS. The server receives listed requests and works with the database, storing everything in one place. That way all users and coaches get the same data. The phone never talks to the database directly, and the server checks every request first this leads to improved security. Maintainability is also better ,because if we need to update something in the backend the mobile application doesn't have to change. This supports the security requirement from phase 2 and also makes the system much easier to maintain.

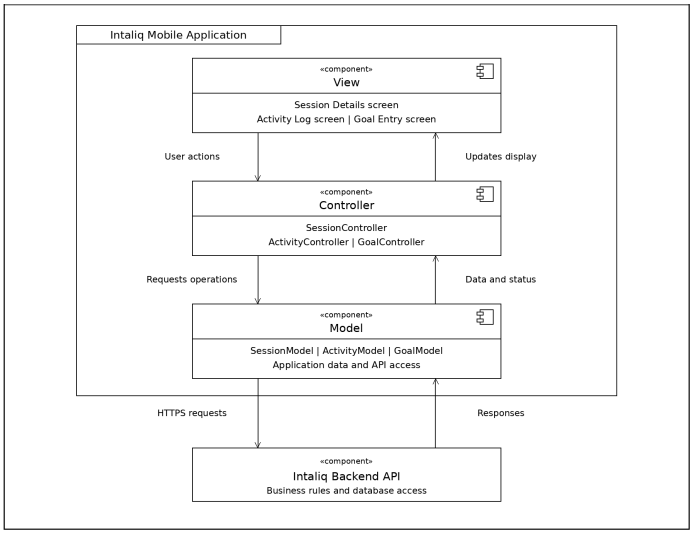
2.5 Mobile Application Architectural Viewpoint

2.5.1 MVC Design View

The application of the Intaliq follows the MVC framework. The model takes care of the application data and interfaces with the backend API while the view handles the display of the data and the inputs of the users, and the controller dictates the users' actions.

Element	Responsibility
Model	Maintain data about sessions, activities and goals. Communicates with the backend API to retrieve and update information.
View	Displays screens for users, collects user input and presents results and error messages.
Controller	Handles actions from the View, coordinates operations through the Model, and updates the View with the results.

MVC Design Diagram



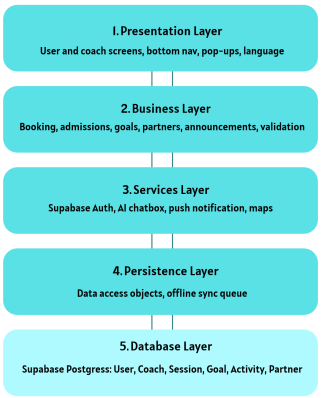
2.5.2 Justification

The MVC architectural pattern is a pattern that divides the architecture into three main blocks: the Model, which handles application data; the View, which handles the user interface; and finally, the Controller, which processes user actions. MVC is an appropriate pattern for the Intaliq application because it separates the application into three main components: the user interface, application data, and control logic. The purpose of this separation is to improve the application's maintainability, extensibility, and testability. The Model manages the data used by the application. The View represents the UI and how the data is presented to users; it also allows users to interact with the application. Finally, the Controller manages and processes actions received from the View and updates the View. For example, when a user books a class or group session, the View is responsible for sending the user's request to the Controller. Then, the Controller processes the request and communicates with the backend through the Model. It also instructs the View to display the output, either a confirmation or an error message.

2.6 Backend Architectural Viewpoint

2.6.1 Layered Architecture Design View

Layered Architecture Design Diagram



2.6.2 Justification

For Intaliq, using a layered architecture for the backend comes with multiple benefits. This architectural approach divides the system into different layers in which each layer has its own responsibilities and performs its own functions and only communicates with its adjacent layer. This approach helps the system be easier to maintain due to changes in one layer not affecting the other layers. Direct access to the database is restricted because requests must pass through the layer above it before this results in advancements in security. An additional benefit is improved testability due to each layer being tested and examined on its own. This architectural approach promotes reusability as the lower layers can be accessed by multiple components of the application. Ultimately, it improves development productivity and efficiency by allowing several different developers to work on different layers at the same time.

3. Detailed Design

3.1 Overview

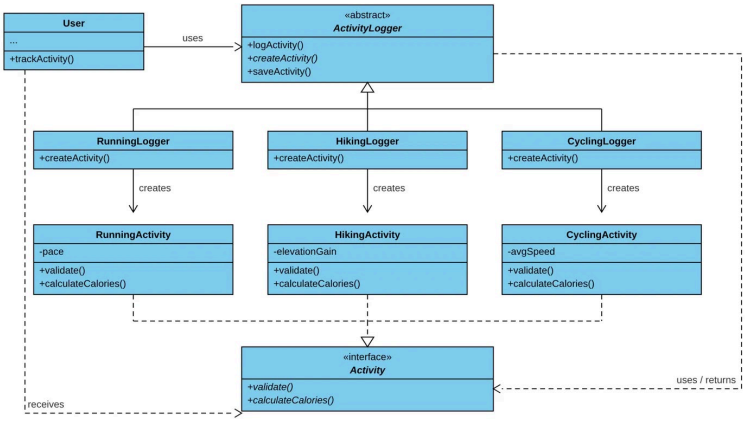
This section presents Intaliq's detailed class-level design using Gang of Four (GoF) design patterns to achieve maintainability, testability, and loose coupling:

- **Factory Method (Creational):** Encapsulates creation logic for exercise activities (Running, Hiking, and Cycling) so new activity types can be added without altering existing code.
- **Observer (Behavioral):** Automatically notifies users and coaches of session updates and booking decisions while keeping session data decoupled from messaging logic.
- **Adapter (Structural):** Integrates third-party services (such as external AI chat services) by adapting incompatible external interfaces into standardized backend calls.

3.2 Factory Method (Creational)

3.2.1 How It Is Applied

The Factory Method design pattern can be applied for Intaliq to log their outdoor activities. This feature allows users to log 3 different types of outdoor activities: running, hiking, and cycling. These activities carry out the same logging process which consists of: validating the activities data, calculating the calories being burnt, and then saving the activity. Yet, each type of activity employs its own approach for validation of data and calculates burnt calories. Rather than relying on enabling the main logging process to determine the corresponding activity object to be created, the task is assigned to a separate logger for each activity type. For example, when a user chooses hiking the HikingLogger creates a HikingActivity object which is utilized by the main logging process without having to know the exact type of activity. This design makes our system much more flexible if we decide to add another activity in the future such as padel by creating its own dedicated activity loggers and classes without changing the current logging process.



A class diagram is provided to visualize how the Factory Method design pattern is applied in Intilaq.

3.2.2 Justification

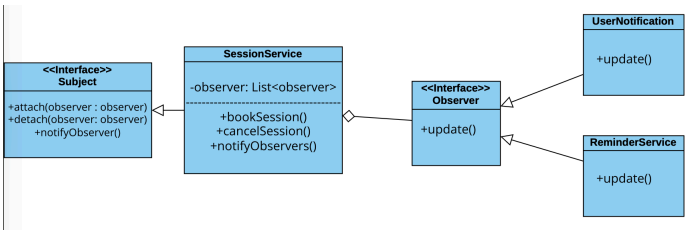
For Intaliq, using the factory method comes with many advantages. That being said, the factory method actually improves testability, while making it easier to maintain specific parts of the code. This method also allows reuse of specific parts of the code. Moreover, we can have the same code working with various existing or newly created product classes. Finally, this method allows development to reach peak efficiency by allowing different developers to work on different parts of the project simultaneously.

3.3 Observer (Behavioral)

3.3.1 How It Is Applied

The Observer design pattern can be applied to Intaliq's notification and update system. The subject is the sessions service, while the observers will be the user notification and reminder services. When an event occurs, for example, a group class is cancelled, the subject can notify all the observers. Then each observer can perform its own independent action. The user notification observer will send notifications to the user and the reminder service may change or modify the reminder schedule. This pattern is useful because it improves flexibility because new response mechanisms like notification reminders can be

modified, added, removed without changing the main session service. This pattern also reduces the dependency between business login and the UI and notification components.



A class diagram is used to visualize how the Observer design pattern can be applied to Intaliq.

3.3.2 Justification

Several events in Intaliq must be communicated to more than one party at the same time. When a coach announces an update to a scheduled session, every admitted member must be informed. As stated in the “Book a Session” scenario, when the coach accepts or declines the booking request, the requesting user needs to be informed about it. In case the class “Session” would be in charge of informing every single user directly, it will get tightly coupled to the notifying operation and any additional notification method would need changes in the class “Session.”

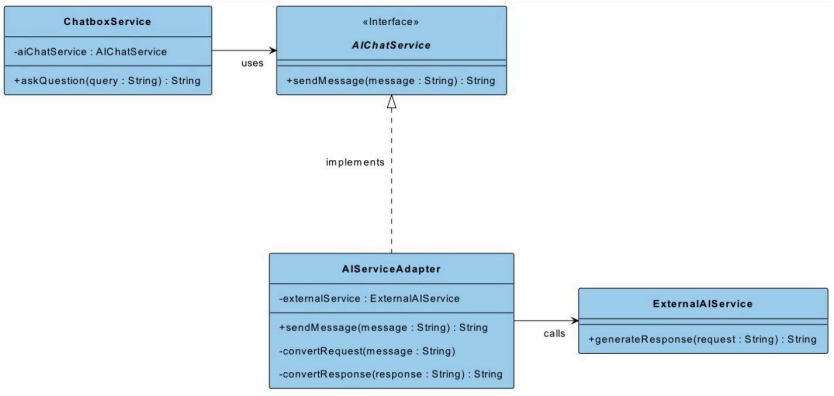
The Observer pattern was chosen because it solves this problem directly. The Session acts as the subject and only needs to announce that a change has occurred. The registered observers, such as the admitted users and the Notifications component, are then updated automatically. This keeps the Session class focused on managing session data rather than on how messages are delivered.

The pattern also supports the quality attributes identified in Phase 2. It promotes usability because users get notifications for any modifications that happen to the session they have joined. At the same time, it facilitates maintainability in that additional observers like email or in-app notification services can be added without changing the current Session class.

3.4 Adapter (Structural)

3.4.1 How It Is Applied

The Adapter design pattern can be applied to our AI Chatbox in Intaliq. In this feature, the user types a fitness related question and eventually gets an answer from an external AI service. The problem is that this external service has its own distinctive way of receiving requests and sending back answers, and it is not the same format our backend uses. So the backend could not talk to it directly. To fix this, our solution was putting an adapter in the middle. The ChatboxService only calls a standard interface named AIChatService, using its sendMessage() method.The AIServiceAdapter implements that interface. When a user asks a question, the adapter will change it to the format the external service understands and sends it over HTTPS. After the service replies the adapter changes the answer back to the format our backend uses, and then the user sees it in the app. This way the ChatboxService doesn't need to know anything about how external service works. If we decide to use a different AI service later on, we just write a new adapter for it and the rest of the backend stays exactly the same.



A class diagram is provided to show how the Adapter design pattern is applied in Intaliq.

3.4.2 Justification

The reason for using the Adapter pattern is the ability to make the ChatboxService in

Intaliq use some external AI service with a unified interface. This way, provider-specific request/response processing logic is decoupled from chatbox logic in the application.

This makes the software more maintainable since any changes in the API of the external service can be processed in AIServiceAdapter. At the same time, it becomes possible to add another AI service provider by implementing the same AIChatService interface; only little modifications will be needed in ChatboxService.

4. Reference

Albugami, Y., Al Rakan, R., Alomran, S., Alghofaili, F., Almuqrin, M., & Alfaris, M. (2026). *Intaliq: Software architecture design* [Project Deliverable 1, SE322]. Software Architecture. https://software-architecture.shoug-tech.com

Albugami, Y., Al Rakan, R., Alomran, S., Alghofaili, F., Almuqrin, M., & Alfaris, M. (2026). *Intaliq: Architecture documentation* [Project Deliverable 2, SE322]. Software Architecture. https://software-architecture.shoug-tech.com

AlOmayrini, D., Alrakan, R., Albagma, W., & Albugami, Y. (2026). *Intaliq 2.0 software requirements specification* [SE311 project document]. Software Architecture. https://software-architecture.shoug-tech.com

5. Glossary

Term	Definition
Activity	An outdoor exercise logged by a user: Running, Hiking, or Cycling
Adapter Pattern	A structural pattern that converts one interface into another that the system expects
API	Application Programming Interface: a defined way for software components to communicate
Architectural Style	A general structure for organizing a system's components, such as Client-Server
Architectural Viewpoint	A perspective used to describe the design for a specific set of stakeholder concerns

Behavioral Pattern	A pattern concerned with how objects communicate and share responsibilities
Client-Server Architecture	A style where clients send requests to a central server that processes them and returns results
Coach	A member of Intaliq's fitness team who creates and manages group sessions
Creational Pattern	A pattern concerned with how objects are created
Design Pattern	A reusable solution to a common problem in software design
Design View	The application of a viewpoint to a specific part of the system
Factory Method Pattern	A creational pattern that lets a dedicated class decide which object type to create
GoF	Gang of Four: Gamma, Helm, Johnson, and Vlissides, authors of the original design patterns book
Goal	A fitness target a user sets for an activity, with a time, calories, and deadline
HTTPS	Hypertext Transfer Protocol Secure: encrypted communication between the app and the server
Observer Pattern	A behavioral pattern where a subject automatically notifies all of its observers when its state changes
Offline Sync	A mobile app feature that stores actions locally and syncs them when the connection returns
Partner	Another user a person connects with to exercise together
SAD	Software Architecture Design document
SDD	Software Design Document
Session	A coach-led group activity with a date, time, location, capacity, and intensity
SRS	Software Requirements Specification
Stakeholder	Anyone with an interest in the system, such as users, coaches, or developers

Structural Pattern	A pattern concerned with how classes and objects are combined into larger structures